

Week_7_Text_Analytics_case_study_with_solutions

April 29, 2026

0.1 Analytics Specializations & Applications - Week 7

0.2 # Text Analytics

Dr Georgiana Nica-Avram - University of Nottingham [mail](#) [web](#)

There is a growing specialization in business analytics that is concerned with Text Analysis and natural language processing (NLP), as it allows businesses to interrogate the mass of unstructured data that exists about their customers, products and interactions - data that far outweighs the “structured” datasets we have considered up to now.

However, analysing such data is non-trivial. Computers are good at understanding rigidly-structured data with limited variety. However, when we deal with unstructured free-form text, things begin to get difficult.

Some of the most common applications of NLP include sentiment analysis, topic modeling, part-of-speech tagging, entity recognition, and so on. The goal of NLP is to develop a set of algorithms so that a company can interact with text written in plain English to use some of the outputs in their businesses processes. We are going to explore how to do this below, in the context of BlueSky posts from a leading publication on emerging technology news and insights.

1 Task 7.1 Setting up your Bluesky account

If you need to scrape private data, search more aggressively, or perform actions (like liking a post), you need a “session”. Start by creating an “App Password” in your Bluesky settings first.

```
[1]: import requests
import pandas as pd
from datetime import datetime, UTC
import time

# Configuration & Authentication
HANDLE = "shreyuu-dev.bsky.social"
    ↪ # replace 'handle' with the account name you chose on Bluesky
APP_PASSWORD = "osip-pbxt-xnfz-52kf"
    ↪ # Generate this in Settings, App Passwords
BASE_URL = "https://bsky.social/xrpc"

def get_session():
```

```

resp = requests.post(
    f"{BASE_URL}/com.atproto.server.createSession",
    json={"identifier": HANDLE, "password": APP_PASSWORD}
)
resp.raise_for_status()
return resp.json()

# The Authenticated Scraper
def scrape_bluesky_full_access(target_handle, max_pages=10):
    session = get_session()
    headers = {"Authorization": f"Bearer {session['accessJwt']}"}

    all_posts = []
    cursor = None

    for page in range(max_pages):
        params = {
            "actor": target_handle,
            "limit": 50,
        }
        ↪ # Authenticated limit is usually higher
        if cursor:
            params["cursor"] = cursor

        url = f"{BASE_URL}/app.bsky.feed.getAuthorFeed"
        response = requests.get(url, params=params, headers=headers)

        if response.status_code == 401:
            ↪ # This means your token expired
            session = get_session()
            headers = {"Authorization": f"Bearer {session['accessJwt']}"}
            continue

        data = response.json()
        feed = data.get('feed', [])

        for item in feed:
            ↪ # These are the post details we are harvesting
            post = item.get('post', {})
            all_posts.append({
                "cid": post.get('cid'),
                "text": post.get('record', {}).get('text'),
                "created_at": post.get('record', {}).get('createdAt'),
                "reply_count": post.get('replyCount'),
                "repost_count": post.get('repostCount'),
                "like_count": post.get('likeCount')
            })

```

```

        cursor = data.get('cursor')
        if not cursor:
            break

        print(f"Scraped {len(all_posts)} posts so far...")
        time.sleep(0.5) # Authenticated requests can be faster

    return pd.DataFrame(all_posts)

df = scrape_bluesky_full_access("technologyreview.com")
↳ # This is the name of the account we are scraping

```

```

Scraped 50 posts so far...
Scraped 100 posts so far...
Scraped 150 posts so far...
Scraped 200 posts so far...
Scraped 250 posts so far...
Scraped 300 posts so far...
Scraped 350 posts so far...
Scraped 400 posts so far...
Scraped 449 posts so far...
Scraped 499 posts so far...

```

[26]: *# In this example, you are scraping posts using keyword searches.*

```

# !pip install atproto
from atproto import Client

import requests
import time
import pandas as pd

def get_session():
    resp = requests.post(
        f"{BASE_URL}/com.atproto.server.createSession",
        json={"identifier": HANDLE, "password": APP_PASSWORD}
    )
    resp.raise_for_status()
    return resp.json()

def search_bluesky_posts(keyword, max_pages=100):
    session = get_session()
    headers = {"Authorization": f"Bearer {session['accessJwt']}" }

    all_posts = []

```

```

cursor = None

for page in range(max_pages):
    params = {
        "q": keyword,
        "limit": 30
    }

    if cursor:
        params["cursor"] = cursor

    url = f"{BASE_URL}/app.bsky.feed.searchPosts"
    response = requests.get(url, params=params, headers=headers)

    # Handle expired token
    if response.status_code == 401:
        session = get_session()
        headers = {"Authorization": f"Bearer {session['accessJwt']}"}
        continue

    response.raise_for_status()
    data = response.json()

    posts = data.get("posts", [])

    for post in posts:
        record = post.get("record", {})

        all_posts.append({
            "cid": post.get("cid"),
            "text": record.get("text"),
            "created_at": record.get("createdAt"),
            "author": post.get("author", {}).get("handle"),
            "reply_count": post.get("replyCount"),
            "repost_count": post.get("repostCount"),
            "like_count": post.get("likeCount")
        })

    cursor = data.get("cursor")
    if not cursor:
        break

    #print(f"Scraped {len(all_posts)} posts so far...")
    time.sleep(0.5) # stay within safe rate limits

return pd.DataFrame(all_posts)

```

```
df = search_bluesky_posts("huggingface") #  
↳ replace "keyword" with your own keyword
```

```
[2]: # Make sure to save your dataframe locally so that you don't have to scrape the  
↳ data all over again.
```

```
df.to_csv(r'...\bluesky_posts.csv', sep = ',', header = True)
```

2 Task 7.2 Basic text statistics

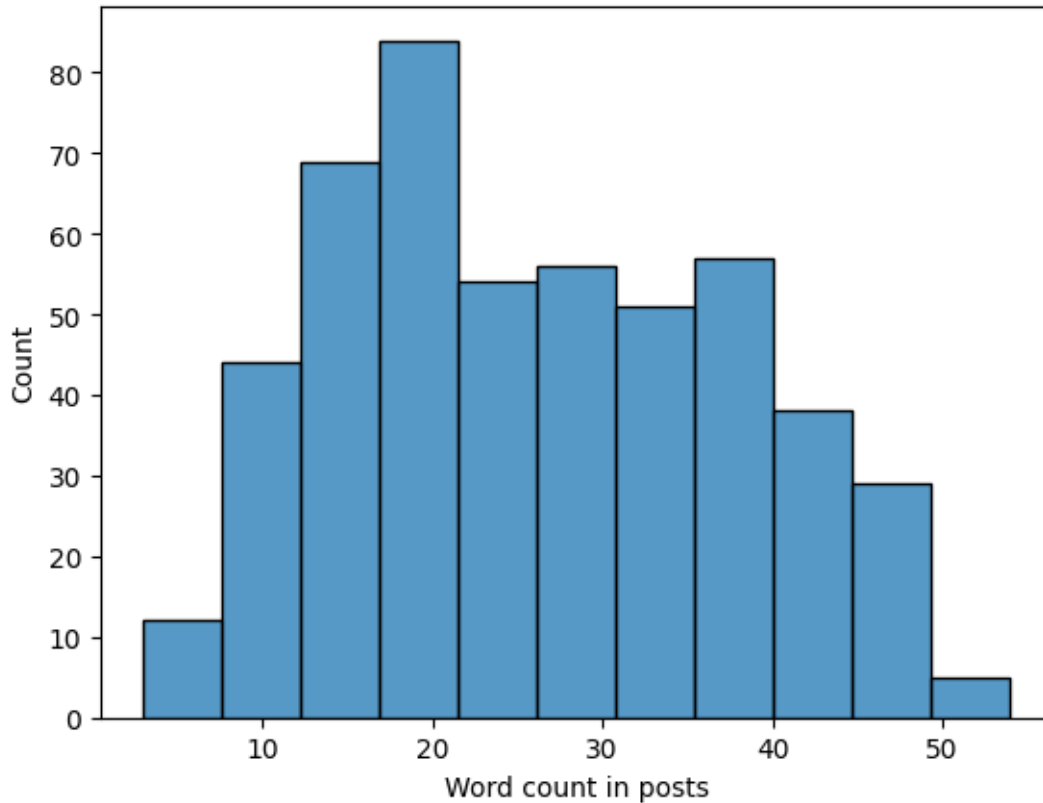
Let's have a look at some basic text statistics, such as the number of words in each post. Micro-blogging platforms such as Bluesky often have short-length posts. This simple statistic will inform your choice of algorithms later on in the analysis.

```
[3]: import seaborn as sns  
import matplotlib.pyplot as plt  
  
df = df.set_index('cid')  
df['post_word_count'] = df['text'].str.split().str.len()  
  
print(f"We have collected {len(df)} posts.")  
print(df['post_word_count'].describe())  
sns.histplot(df['post_word_count'])  
plt.xlabel('Word count in posts')  
plt.show()
```

We have collected 499 posts.

count	499.000000
mean	26.144289
std	11.554099
min	3.000000
25%	16.500000
50%	25.000000
75%	36.000000
max	54.000000

Name: post_word_count, dtype: float64



3 Task 7.3 Advanced text statistics

We start by splitting our posts into sentences and words, and removing unnecessary characters such as punctuation.

```
[6]: # Gensim is a Python library for topic modelling, document indexing and
      ↪ similarity retrieval with large corpora. The first step is to install gensim
      ↪ library dependencies.
      # !pip install gensim
      from gensim import *
      import gensim.utils
      from gensim.utils import simple_preprocess
      from nltk.stem import WordNetLemmatizer
      from collections import defaultdict
      from pprint import pprint
      from six import iteritems
      from collections import Counter
      from gensim.models import ldaseqmodel, Phrases, Word2Vec
      from gensim.models.phrases import Phraser
```

```

documents= []
for line in df['text']:
    documents.append(line)

texts= []
for document in documents:
    text= gensim.utils.simple_preprocess(str(document), deacc= True)
    ↪# deacc removes punctuation
    texts.append(text)

dictionary= corpora.Dictionary(texts)
    ↪# The Gensim dictionary contains the mapping of all words, a.k.a tokens, to
    ↪their unique integer id
corpus= [dictionary.doc2bow(d) for d in texts]

```

[7]: # The next step is to visualise the most common 30 words

```

from collections import Counter

print('Number of unique tokens: %d' % len(dictionary))
print('Number of documents: %d' % len(corpus))

# Frequency table
freq = Counter(t for text in texts for t in text)
word_freq = (
    pd.DataFrame(freq.items(), columns=['token', 'freq'])
    .query('token in @dictionary.token2id')
)

# Top 30 plot
top30 = word_freq.nlargest(30, 'freq')

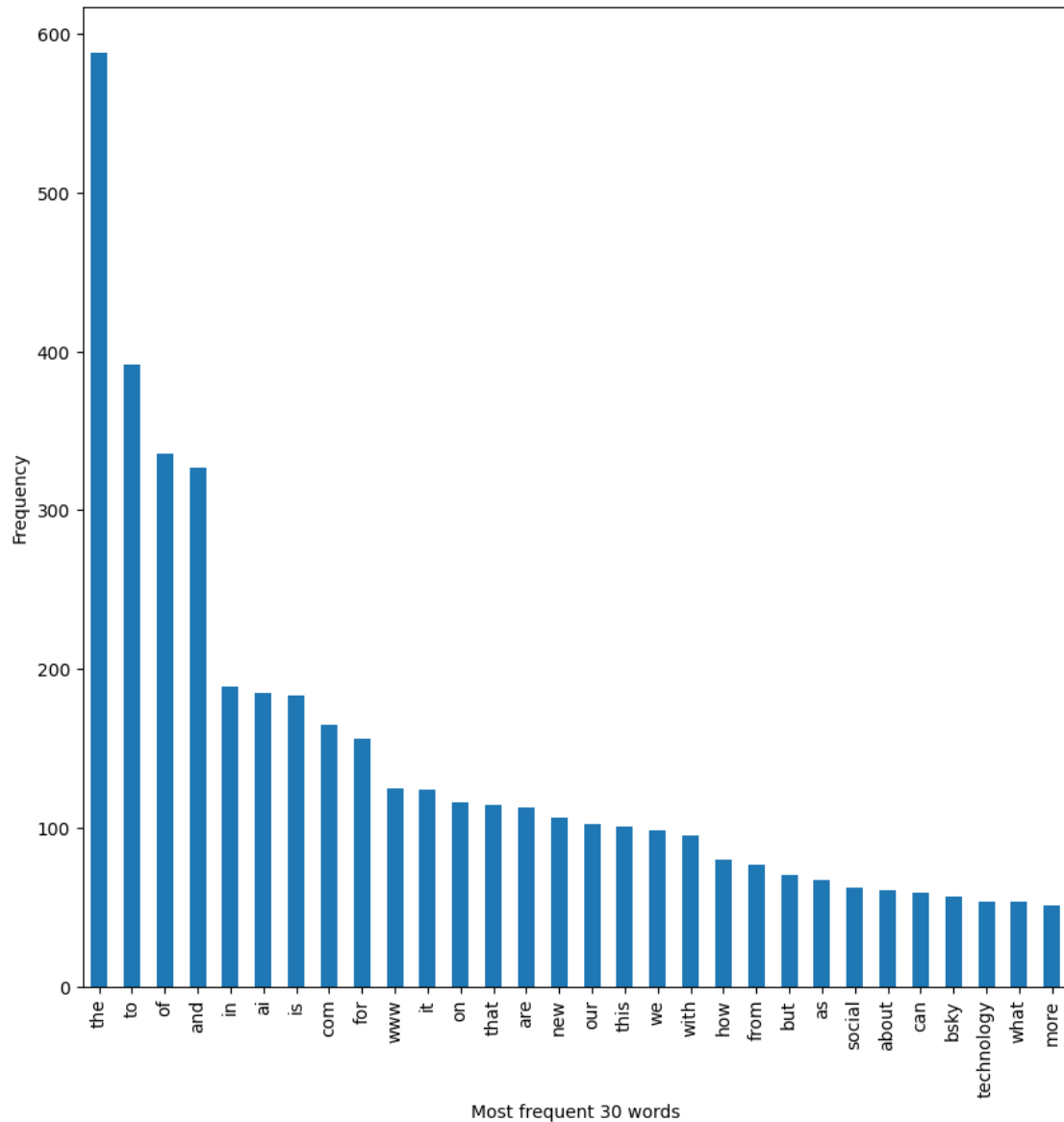
top30.plot(kind='bar', x='token', y='freq', legend=False, figsize=(10,10))
plt.xlabel('Most frequent 30 words')
plt.ylabel('Frequency')
plt.xticks(rotation=90)

# Rank all tokens
sorted_freq = word_freq.sort_values('freq', ascending=False).assign(rank=lambda
    ↪d: range(1, len(d)+1))

```

Number of unique tokens: 3262

Number of documents: 499



```
[8]: # Log-log plots of word rank versus frequency visualize Zipf's law, which
      ↪ states that word frequency is inversely proportional to its rank.
      # On a logarithmic scale, this relationship appears as a straight line with a
      ↪ slope of approximately -1, where a few words are extremely common, and most
      ↪ are rare.
      # As we saw earlier, the most common words in this corpus are articles (stop
      ↪ words) - often not informative for understanding the topics discussed in the
      ↪ posts.
      # We will deal with stop words in the next tasks.
```

```
import numpy as np
```

```

from scipy import stats

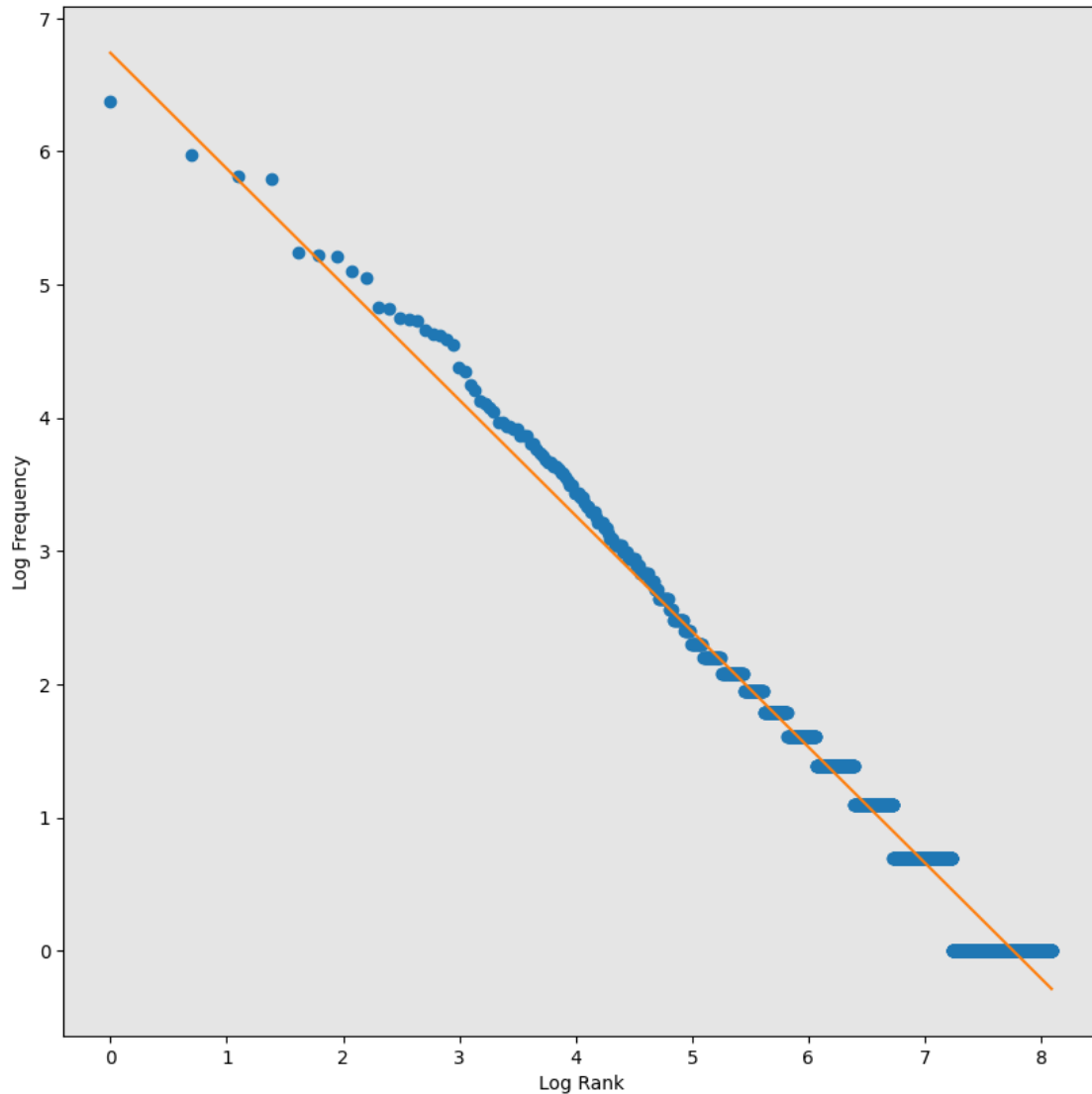
log_rank= np.log(sorted_freq['rank'])
log_frequency= np.log(sorted_freq['freq'])

slope, intercept, r_value, p_value, std_err= stats.linregress(log_rank,
    ↪log_frequency)
line= slope*log_rank+intercept

fig, ax= plt.subplots(figsize= (10, 10))
plt.plot(log_rank, log_frequency, 'o', log_rank, line)
ax= plt.gca()
ax.set_facecolor((0.898, 0.898, 0.898))
fig= plt.gcf()
ax.set_xlabel('Log Rank')
ax.set_ylabel('Log Frequency')
plt.show()

print('Slope', np.round(slope, 2), 'Intercept', np.round(intercept, 2),
    ↪'r-value', np.round(r_value, 2), 'r^2 value', np.round(r_value**2, 2),
    ↪'p-value', np.round(p_value, 4), 'Standard error', np.round(std_err, 2))

```



Slope -0.87 Intercept 6.74 r-value -0.98 r^2 value 0.96 p-value 0.0 Standard error 0.0

4 Task 7.4 Topic modeling with BERTopic

Imagine reading 1,000 customer reviews and instantly seeing themes emerge — not just based on repetitive words, but actual concepts. That's BERTopic.

BERTopic is a modern out of the box topic modeling framework that addresses many limitations of traditional approaches by:

1. using transformer-based embeddings (like BERT) to understand the semantic meaning of documents

2. representing documents using Class-based TF-IDF (c-TF-IDF) to highlight important words per topic by comparing them against the entire dataset, improving topic interpretability
3. clustering them based on their context rather than just word frequency

An important advantage of BERTopic is that it requires almost no preprocessing of the given texts. Unlike LDA, however, BERTopic assumes each document contains only one topic.

```
[9]: # !pip install bertopic
from sentence_transformers import SentenceTransformer
import bertopic
from bertopic import BERTopic
from bertopic.representation import KeyBERTInspired

# Load a custom embedding model
# all-MiniLM-L6-v2 is a fast sentence-transformer model that maps sentences and
# paragraphs into a 384-dimensional dense vector space.
# It is primarily used for semantic search, text clustering, and similarity
# matching, offering high performance for tasks requiring semantic
# understanding.
embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

# Fine-tune topic representation
# In BERTopic, there are a number of different topic representations that we
# can choose from.
# A great start is KeyBERTInspired, which for many users increases the
# coherence and reduces stopwords from the resulting topic representations.
representation_model = KeyBERTInspired()

# Pass it into BERTopic
posts = df['text'].to_list()

topic_model = BERTopic(embedding_model=embedding_model, min_topic_size = 10,
# representation_model = representation_model)
topics, probs = topic_model.fit_transform(posts)
```

```
/Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

```
[10]: # After generating topics and their probabilities, we can access the frequent
# topics that were generated, including a topic name and most representative
# posts for that topic.
topic_labels = topic_model.generate_topic_labels(nr_words = 3, topic_prefix =
# False, word_length = 15, separator = "_")
topic_model.set_topic_labels(topic_labels)
```

```
# Topic -1 refers to all outliers and should typically be ignored.
topic_info = topic_model.get_topic_info()
print(topic_info)
```

	Topic	Count	Name \
0	-1	3	-1_sources_others_cautious_we
1	0	416	0_ai_technology_research_technologies
2	1	80	1_ai_breakthrough_technology_research

	CustomName \
0	sources_others_cautious
1	ai_technology_research
2	ai_breakthrough_technology

	Representation \
0	[sources, others, cautious, we, openai, rumors...
1	[ai, technology, research, technologies, scien...
2	[ai, breakthrough, technology, research, techn...

	Representative_Docs
0	["Our focus is really on what we can do to pul...
1	[The latest edition of our State of AI collabo...
2	[This feature from @antonioregalado.bsky.socia...

```
[11]: !pip install WordCloud
```

```
Requirement already satisfied: WordCloud in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (from WordCloud) (2.3.4)
Requirement already satisfied: pillow in /Users/shreyu/VSCODE/junk/UoN-Business-
Analytics/.venv/lib/python3.14/site-packages (from WordCloud) (12.0.0)
Requirement already satisfied: matplotlib in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (from WordCloud) (3.10.7)
Requirement already satisfied: contourpy>=1.0.1 in
/Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-
packages (from matplotlib->WordCloud) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (from
matplotlib->WordCloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-
packages (from matplotlib->WordCloud) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-
packages (from matplotlib->WordCloud) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (from
```

matplotlib->WordCloud) (25.0)
Requirement already satisfied: pyparsing>=3 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from matplotlib->WordCloud) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from matplotlib->WordCloud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from python-dateutil>=2.7->matplotlib->WordCloud) (1.17.0)

```
[12]: # An informative way of visualising topic distributions over words is through wordclouds
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import math

# Get all valid topic IDs
top_topics = topic_model.get_topic_info()['Topic'].tolist()

cols = 3
rows = math.ceil(len(top_topics) / cols)

fig, axes = plt.subplots(rows, cols)
axes = axes.flatten()

for i, topic_id in enumerate(top_topics):
    topic_words = {word: score for word, score in topic_model.get_topic(topic_id)}
    wc = WordCloud(background_color="white").generate_from_frequencies(topic_words)

    axes[i].imshow(wc, interpolation="bilinear")
    axes[i].set_title(f"Topic {topic_id}")
    axes[i].axis("off")

plt.tight_layout()
plt.show()
```



```
[13]: # Find the most relevant topic for the word "AI".
topic_ids, similarity_scores = topic_model.find_topics("AI", top_n = 1)

# Output the results
print("Topic IDs: ", topic_ids, "Similarity Scores: ", similarity_scores)

# Inspect the topic's keywords
topic_keywords = topic_model.get_topic(topic_ids[0])
print(f"Keywords for topic {topic_ids[0]}: {topic_keywords}")
```

```
Topic IDs: [0] Similarity Scores: [np.float32(0.5501696)]
Keywords for topic 0: [('ai', np.float32(0.51179546)), ('technology',
np.float32(0.30920318)), ('research', np.float32(0.3087286)), ('technologies',
np.float32(0.30316594)), ('scientists', np.float32(0.2815456)), ('industry',
np.float32(0.27915305)), ('tech', np.float32(0.2717129)), ('technologyreview',
np.float32(0.25651836)), ('companies', np.float32(0.24086933)), ('openai',
np.float32(0.22902578))]
```

Topics resulting from topic modeling are often evaluated in terms of coherence.

Topic coherence is simply a proxy for “does this topic have a clear, singular theme?” Though the consensus is that high coherence score is useless if the words don’t make sense to a human.

Two foundational readings for understanding how coherence metrics are used in Gensim and BERTopic include:

[Coherence models](#)

[Exploring the Space of Topic Coherence Measures](#)

```
[14]: # Using .get_document_info, we can also extract information on a document,
      ↪ level, such as their corresponding topics, probabilities and whether they
      ↪ are representative documents for a topic:
document_info = topic_model.get_document_info(posts)
print(document_info.head(10))
```

	Document	Topic \
0	Super disappointed/jealous I never thought to ...	0
1	See MIT Technology Review’s highly subjective ...	0
2	There are plenty of reasons that even the carl...	0
3	Skiers in the know follow OpenSnow and won’t b...	0
4	SES AI is hoping for a new life after more tha...	0
5	Axiom Math is giving away a powerful new AI to...	0
6	This research could help provide a tool to neu...	1
7	New research can’t yet say whether AI causes d...	0
8	Animal welfare advocates and AI researchers ga...	0
9	Freelance journalists and writers: @technology...	0

Name	CustomName \
------	--------------

0	0_ai_technology_research_technologies	ai_technology_research
1	0_ai_technology_research_technologies	ai_technology_research
2	0_ai_technology_research_technologies	ai_technology_research
3	0_ai_technology_research_technologies	ai_technology_research
4	0_ai_technology_research_technologies	ai_technology_research
5	0_ai_technology_research_technologies	ai_technology_research
6	1_ai_breakthrough_technology_research	ai_breakthrough_technology
7	0_ai_technology_research_technologies	ai_technology_research
8	0_ai_technology_research_technologies	ai_technology_research
9	0_ai_technology_research_technologies	ai_technology_research

	Representation \
0	[ai, technology, research, technologies, scien...
1	[ai, technology, research, technologies, scien...
2	[ai, technology, research, technologies, scien...
3	[ai, technology, research, technologies, scien...
4	[ai, technology, research, technologies, scien...
5	[ai, technology, research, technologies, scien...
6	[ai, breakthrough, technology, research, techn...
7	[ai, technology, research, technologies, scien...
8	[ai, technology, research, technologies, scien...
9	[ai, technology, research, technologies, scien...

	Representative_Docs \
0	[The latest edition of our State of AI collabo...
1	[The latest edition of our State of AI collabo...
2	[The latest edition of our State of AI collabo...
3	[The latest edition of our State of AI collabo...
4	[The latest edition of our State of AI collabo...
5	[The latest edition of our State of AI collabo...
6	[This feature from @antonioregalado.bsky.socia...
7	[The latest edition of our State of AI collabo...
8	[The latest edition of our State of AI collabo...
9	[The latest edition of our State of AI collabo...

	Top_n_words	Probability \
0	ai - technology - research - technologies - sc...	1.000000
1	ai - technology - research - technologies - sc...	1.000000
2	ai - technology - research - technologies - sc...	1.000000
3	ai - technology - research - technologies - sc...	0.935740
4	ai - technology - research - technologies - sc...	0.992844
5	ai - technology - research - technologies - sc...	1.000000
6	ai - breakthrough - technology - research - te...	0.961891
7	ai - technology - research - technologies - sc...	1.000000
8	ai - technology - research - technologies - sc...	1.000000
9	ai - technology - research - technologies - sc...	1.000000

Representative_document

```

0           False
1           False
2           False
3           False
4           False
5           False
6           False
7           False
8           False
9           False

```

5 Task 7.5 Topic modeling with LDA

Before we test an alternative topic modeling approach, we need to get our data into shape. As we saw earlier, many of the words in the corpus are not informative. So we start by cleaning the posts.

5.1 Task 7.5.1 Cleaning the posts

```

[16]: import re
import nltk
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('stopwords')
nltk.download('wordnet')
import string
from nltk.corpus import stopwords
nltk.download('stopwords')

from nltk.stem import WordNetLemmatizer
from nltk import word_tokenize, sent_tokenize
from textblob import Word

# Cleaning the posts by converting to lower case and removing punctuation and
↳ digits, replacing accentuated characters
def cleanText(text):
    text= re.sub(r"[^\w\s]", " ", str(text)) #remove punctuation
    text= re.sub(r"\d+", "", str(text))      #remove digits
    text= re.sub(r"\b[a-zA-Z]\b", "", str(text)) #remove one-letter words
    text= text.lower()
    text= re.sub(u"[\u00c0\u00c1\u00c2\u00c3\u00c4\u00c5]", 'a', str(text))
    text= re.sub(u"[\u00e0\u00e1\u00e2\u00e3\u00e4\u00e5]", 'e', str(text))
    text= re.sub(u"[\u00f0\u00f1\u00f2\u00f3\u00f4\u00f5]", 't', str(text))
    return text

# Stopwords
stop= stopwords.words('english')

# Lemmatization - reduces inflected words to their base or dictionary form,
↳ known as a lemma.

```

```

lemmatizer= WordNetLemmatizer()

df['text_clean']= df['text'].apply(cleanText)
df['text_clean']= df['text_clean'].apply(lambda x: ' '.join([word for word in x.
    ↪split() if word not in (stop)]))
df['text_clean']= df['text_clean'].apply(lambda x: ' '.join([Word(word).
    ↪lemmatize() for word in x.split()]))

# Data lists
documents_clean= []
for line in df['text_clean']:
    documents_clean.append(line)

texts_clean= []
for document in documents_clean:
    text= gensim.utils.simple_preprocess(str(document), deacc= True) #deacc_
    ↪removes punctuation
    texts_clean.append(text)

```

```

[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /Users/shreyu/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
[nltk_data] Downloading package stopwords to
[nltk_data] /Users/shreyu/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /Users/shreyu/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] /Users/shreyu/nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

We cleaned our posts of redundant words and characters. Now it's time to convert them in a numeric format. We use the “bag of words” approach and describe each post with no regards to the ordering of the words, but simply their counts. This turns out to still be a very powerful and flexible approach, allowing us to compare the similarity between two pieces of text.

To do this we use a “Vector Space” representation of a document. First we consider every word that can appear in a corpus (i.e., across all the BlueSky posts we have) as a feature in its own right. We then simply assign the value of that feature for any individual document, as the number of times that word appears. This means if we have M words in our “dictionary” of words that appear across the corpus, each document can be represented by M features, and we are back in our traditional home of seeing a document as a point in an M -dimensional space.

By way of example consider an example (highly condensed!) universe where there are only 3 possible words: “Apple”, “Orange” and “Banana”. Every sentence of document is made out of them. In this case every document can be represented in a 3 dimensional space. Each word is an axis of that space:

Apple -> [1,0,0]

Banana -> [0,1,0]

Orange -> [0,1,0]

Then a sentence or a document is simply a linear combination of these vectors, where the number of times that word appears is the coefficient along that dimension. For example consider:

dM = "Banana Apple Orange" -> [1, 1, 1]

d1 = "Banana Apple Banana Banana Banana Apple" -> [2, 4, 0]

d2 = "Banana Orange Banana Banana Orange Banana" -> [0, 4, 2]

d3 = "Apple Orange Orange Apple" -> [2, 0, 2]

d4 = "Apple Banana Apple Banana" -> [2, 2, 0]

d5 = "Banana Apple Banana Banana Orange Banana" -> [1, 4, 1]

```
[17]: # Dictionary and corpus after cleaning
dictionary_clean= corpora.Dictionary(texts_clean) #dictionary
dictionary_clean.filter_extremes(no_below= 3, no_above= 0.95)
corpus_clean= [dictionary_clean.doc2bow(d) for d in texts_clean]

print('Number of unique tokens: %d' % len(dictionary_clean))
print('Number of documents: %d' % len(corpus_clean))

line_clean= []
for line in df['text_clean']:
    documents_clean.append(line)
    line_clean.append(len(line))
print('Maximum number of words in a document: %d' % max(line_clean))
print('Mean number of words in a document: %d' % np.mean(line_clean))
print('Minimum number of words in a document: %d' % min(line_clean))

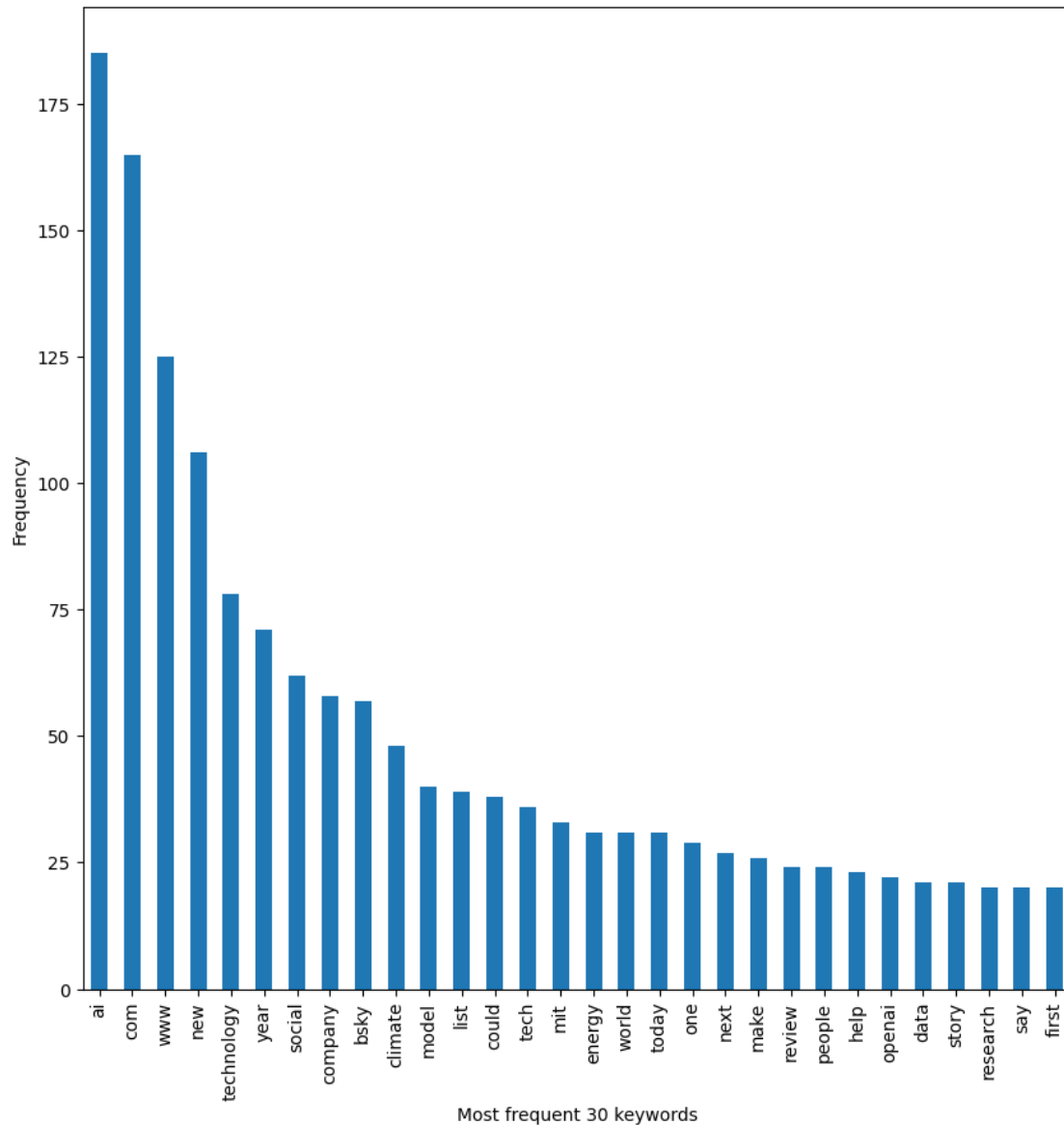
#-----
# Frequency table
freq = Counter(t for text in texts_clean for t in text)
word_freq = (
    pd.DataFrame(freq.items(), columns=['token', 'freq'])
    .query('token in @dictionary.token2id')
)

# Top 30 plot
top30 = word_freq.nlargest(30, 'freq')

top30.plot(kind='bar', x='token', y='freq', legend=False, figsize=(10,10))
plt.xlabel('Most frequent 30 keywords')
plt.ylabel('Frequency')
plt.xticks(rotation=90)
```

Number of unique tokens: 711
Number of documents: 499
Maximum number of words in a document: 228
Mean number of words in a document: 119
Minimum number of words in a document: 14

```
[17]: (array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
            17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]),
      [Text(0, 0, 'ai'),
       Text(1, 0, 'com'),
       Text(2, 0, 'www'),
       Text(3, 0, 'new'),
       Text(4, 0, 'technology'),
       Text(5, 0, 'year'),
       Text(6, 0, 'social'),
       Text(7, 0, 'company'),
       Text(8, 0, 'bsky'),
       Text(9, 0, 'climate'),
       Text(10, 0, 'model'),
       Text(11, 0, 'list'),
       Text(12, 0, 'could'),
       Text(13, 0, 'tech'),
       Text(14, 0, 'mit'),
       Text(15, 0, 'energy'),
       Text(16, 0, 'world'),
       Text(17, 0, 'today'),
       Text(18, 0, 'one'),
       Text(19, 0, 'next'),
       Text(20, 0, 'make'),
       Text(21, 0, 'review'),
       Text(22, 0, 'people'),
       Text(23, 0, 'help'),
       Text(24, 0, 'openai'),
       Text(25, 0, 'data'),
       Text(26, 0, 'story'),
       Text(27, 0, 'research'),
       Text(28, 0, 'say'),
       Text(29, 0, 'first')])
```



5.2 Task 7.5.2 Sentiment analysis

Now let's use our preprocessed posts to do some sentiment analysis.

Polarity measures the emotional tone of text, classifying it as positive, negative, or neutral. It typically uses a score ranging from -1.0 (very negative) to +1.0 (very positive), with 0 indicating neutral.

TextBlob's subjectivity score measures the amount of personal opinion, emotion, or judgment in a text, ranging from 0.0 (very objective/factual) to 1.0 (very subjective/opinion-based).

These metrics are crucial for analyzing user reviews, social media, and market trends.

```
[18]: import textblob
      from textblob import TextBlob

      # We calculate the polarity and subjectivity of each post
      def detect_pol(text):
          return TextBlob(text).sentiment.polarity
      df['polarity'] = df['text_clean'].apply(detect_pol)

      def detect_sub(text):
          return TextBlob(text).sentiment.subjectivity
      df['subjectivity'] = df['text_clean'].apply(detect_sub)
```

```
[19]: df['polarity'].describe()
```

```
[19]: count      499.000000
      mean        0.071225
      std         0.200441
      min        -1.000000
      25%         0.000000
      50%         0.040278
      75%         0.178680
      max         1.000000
      Name: polarity, dtype: float64
```

```
[20]: df['subjectivity'].describe()
```

```
[20]: count      499.000000
      mean        0.370583
      std         0.258201
      min         0.000000
      25%         0.192424
      50%         0.391631
      75%         0.510417
      max         1.000000
      Name: subjectivity, dtype: float64
```

5.3 Task 7.5.3: Topic modeling with LDA

In previous sessions, we used LDA to identify “shopping missions” from the items found in shopping baskets. We extend the idea of “bag of words” to BlueSky to represent in vector space the posts we collected. Unlike BERTopic, LDA gives better results on preprocessed text, and passes it on to gensim’s LdaModel which handles corpora of text very well.

Another difference is that LDA is a probabilistic model that assumes each topic is a mixture over an underlying set of words, and each document is a mixture of over a set of topic probabilities (not just one topic).

```
[21]: from gensim.models import LdaModel
```

```
tfidf= models.TfidfModel(corpus_clean)
tfidf_corpus= tfidf[corpus_clean]
```

```
lda_model= gensim.models.ldamodel.LdaModel(corpus= corpus_clean,
                                             id2word= dictionary_clean,
                                             num_topics= 6,
                                             random_state= 42,
                                             update_every= 1,
                                             chunksize= 100,
                                             passes= 10,
                                             alpha= 5/6,
                                             per_word_topics= True)
```

```
[22]: !pip install pyLDavis
```

```
import pyLDavis
```

```
import pyLDavis.gensim # don't skip this
```

```
pyLDavis.enable_notebook()
```

```
vis= pyLDavis.gensim.prepare(lda_model, corpus_clean, dictionary_clean)
vis
```

Collecting pyLDavis

Downloading pyLDavis-3.4.1-py3-none-any.whl.metadata (4.2 kB)

Requirement already satisfied: numpy>=1.24.2 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from pyLDavis) (2.3.4)

Requirement already satisfied: scipy in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from pyLDavis) (1.16.3)

Requirement already satisfied: pandas>=2.0.0 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from pyLDavis) (2.3.3)

Requirement already satisfied: joblib>=1.2.0 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from pyLDavis) (1.5.2)

Requirement already satisfied: jinja2 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from pyLDavis) (3.1.6)

Collecting numexpr (from pyLDavis)

Downloading numexpr-2.14.1-cp314-cp314-macosx_11_0_arm64.whl.metadata (9.0 kB)

Collecting fancy (from pyLDavis)

Downloading fancy-2.0-py2.py3-none-any.whl.metadata (5.9 kB)

Requirement already satisfied: scikit-learn>=1.0.0 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from pyLDavis) (1.7.2)

Requirement already satisfied: gensim in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from pyLDavis) (4.4.0)

Requirement already satisfied: setuptools in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-packages (from pyLDavis) (80.9.0)

Requirement already satisfied: python-dateutil>=2.8.2 in /Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-

```

packages (from pandas>=2.0.0->pyLDAvis) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (from
pandas>=2.0.0->pyLDAvis) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (from
pandas>=2.0.0->pyLDAvis) (2025.2)
Requirement already satisfied: six>=1.5 in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (from python-
dateutil>=2.8.2->pandas>=2.0.0->pyLDAvis) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-
packages (from scikit-learn>=1.0.0->pyLDAvis) (3.6.0)
Requirement already satisfied: smart_open>=1.8.1 in
/Users/shreyu/VSCODE/junk/UoN-Business-Analytics/.venv/lib/python3.14/site-
packages (from gensim->pyLDAvis) (7.5.1)
Requirement already satisfied: wrapt in /Users/shreyu/VSCODE/junk/UoN-Business-
Analytics/.venv/lib/python3.14/site-packages (from
smart_open>=1.8.1->gensim->pyLDAvis) (2.1.2)
Requirement already satisfied: MarkupSafe>=2.0 in /Users/shreyu/VSCODE/junk/UoN-
Business-Analytics/.venv/lib/python3.14/site-packages (from jinja2->pyLDAvis)
(3.0.3)
Downloading pyLDAvis-3.4.1-py3-none-any.whl (2.6 MB)
      2.6/2.6 MB
21.6 MB/s  0:00:00
Downloading funcy-2.0-py2.py3-none-any.whl (30 kB)
Downloading numexpr-2.14.1-cp314-cp314-macosx_11_0_arm64.whl (152 kB)
Installing collected packages: funcy, numexpr, pyLDAvis
      3/3
[pyLDAvis]2/3 [pyLDAvis]
Successfully installed funcy-2.0 numexpr-2.14.1 pyLDAvis-3.4.1

```

```

[22]: PreparedData(topic_coordinates=          x          y topics cluster
Freq
topic
0      0.228667  0.208540          1          1 20.480297
5     -0.216348  0.011800          2          1 19.121580
4      0.050048 -0.215859          3          1 16.518560
1      0.001356 -0.141835          4          1 15.140086
3     -0.187808  0.164099          5          1 14.878607
2      0.124086 -0.026745          6          1 13.860871, topic_info=
Term      Freq      Total Category logprob loglift
64         com 144.000000 144.000000 Default 30.0000 30.0000
74         www 110.000000 110.000000 Default 29.0000 29.0000
0          ai 168.000000 168.000000 Default 28.0000 28.0000
13  technology 67.000000 67.000000 Default 27.0000 27.0000
7         social 53.000000 53.000000 Default 26.0000 26.0000

```

```

..      ...      ...      ...      ...      ...
307      still      8.304078      14.983276      Topic6      -4.5114      1.3859
30      new      13.061225      87.526859      Topic6      -4.0585      0.0738
177      today      8.632747      31.688821      Topic6      -4.4726      0.6757
64      com      9.806986      144.595410      Topic6      -4.3451      -0.7147
74      www      6.594495      110.415623      Topic6      -4.7420      -0.8419

```

```

[267 rows x 6 columns], token_table=      Topic      Freq      Term
term
533      5      0.955345      administration
179      5      0.113496      agency
179      6      0.794474      agency
81      1      0.474654      agent
81      4      0.474654      agent
...      ...      ...      ...
74      4      0.009057      www
74      6      0.063397      www
145      2      0.933866      year
145      4      0.050938      year
526      2      0.919738      young

```

```

[282 rows x 3 columns], R=30, lambda_step=0.01, plot_opts={'xlab': 'PC1',
'ylab': 'PC2'}, topic_order=[1, 6, 5, 2, 4, 3])

```

```

[23]: num_topics = lda_model.num_topics

cols = 3
rows = math.ceil(num_topics / cols)

fig, axes = plt.subplots(rows, cols, figsize=(10, 10))
axes = axes.flatten()

for i in range(num_topics):
    topic_words = dict(lda_model.show_topic(i, topn=100))

    wc = WordCloud(background_color='white').
    ↪generate_from_frequencies(topic_words)

    axes[i].imshow(wc, interpolation='bilinear')
    axes[i].set_title(f"Topic {i}")
    axes[i].axis("off")

plt.tight_layout()
plt.show()

```